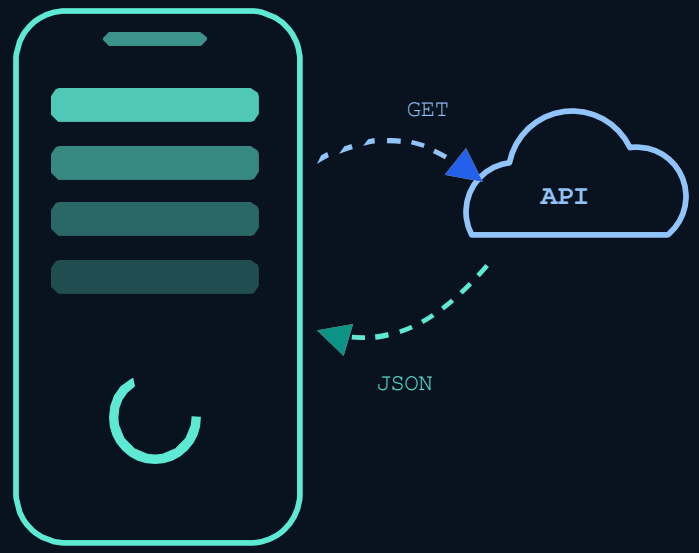
TEMA 4

APIs remotas e processamento assíncrono

Da chamada HTTP ao ecrã — sem bloquear a interface e sem falhar quando a rede falha.



GRUPO
8

DATA
3 de novembro de 2026

CURSO
Engenharia Informática · UBI

ROTEIRO

O que vamos ver

01 **Fundamentos**
HTTP · REST · JSON e serialização · GraphQL

02 **Rede em Android**
Arquitetura · OkHttp · Retrofit

03 **Processamento assíncrono**
Threads · Coroutines · ExecutorService (Java) · LiveData e Flow

04 **Robustez**
Estado da UI · Erros · Tempos de espera · Cache · Indicadores de progresso

05 **Demonstração**
App "Locais IPMA" com a API aberta do IPMA

PERGUNTA-GUIA

Como é que uma app vai buscar dados a um servidor e os mostra numa lista **sem congelar a interface,** e o que faz quando a rede **falha**?

MOTIVAÇÃO

A rede é lenta, imprevisível — e proibida na main thread

16,7 ms
orçamento de cada frame a 60 Hz

5 s
sem responder a um toque → diálogo **ANR** ("a app não responde")

`NetworkOnMainThreadException`
o Android recusa pedidos de rede na main thread (desde a API 11)

✗ **Pedido feito na main thread**



✓ **Pedido assíncrono**



cada bloco = 1 frame (16,7 ms a 60 Hz)

Regra: a **main thread** só desenha e reage a eventos. Rede, disco e cálculo pesado vão para **outra thread** — e o resultado volta à main thread para atualizar a UI.

HTTP: um pedido, uma resposta

Pedido — método + caminho · cabeçalhos · (corpo)

```
GET /open-data/distrits-islands.json HTTP/1.1
Host: api.ipma.pt
Accept: application/json
User-Agent: LocaisIPMA/1.0 (Android)
```

Resposta — código · cabeçalhos · corpo (exemplo simplificado)

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=600

{"owner":"IPMA","country":"PT",
 "data":[{"globalIdLocal":1050200,
          "local":"Castelo Branco", …}]}
```

2xx sucesso
200 OK · 201 Created · 204 No Content

3xx redireção / cache
301 · 304 Not Modified

4xx erro do cliente
400 · 401 · 403 · 404 · 429

5xx erro do servidor
500 · 502 · 503

HTTP não guarda estado entre pedidos. Em Android usa-se **HTTPS**: desde o Android 9, o tráfego em claro (http://) está bloqueado por omissão. A app precisa da permissão `INTERNET` no manifesto.

APIs REST: recursos, verbos e representações

- **Recursos** identificados por URI — `/tarefas/42`
- **Verbos HTTP** com significado — ler, criar, substituir, apagar
- **Sem estado** — cada pedido leva tudo o que precisa (ex.: token)
- **Representações** — normalmente JSON
- **Cache** controlada por cabeçalhos (`Cache-Control`, `ETag`)

MÉTODO	EXEMPLO	SEGURO	IDEMPOTENTE
GET	`GET /tarefas/42`	✓	✓
POST	`POST /tarefas`	✗	✗
PUT	`PUT /tarefas/42`	✗	✓
PATCH	`PATCH /tarefas/42`	✗	✗ *
DELETE	`DELETE /tarefas/42`	✗	✓

Seguro: não altera o servidor. **Idempotente**: repetir dá o mesmo resultado — logo, pode repetir-se depois de um timeout. * depende da implementação.

A API que vamos usar — IPMA open data (só GET, sem chave)

GET `/open-data/distrits-islands.json` — 35 locais

GET `/open-data/forecast/meteorology/cities/daily/{id}.json`

GET `/open-data/weather-type-classe.json` — tipos de tempo

JSON e serialização: do texto para objetos Kotlin

Resposta real do IPMA (Castelo Branco)

```
{
  "owner": "IPMA", "country":"PT",
  "data": [{
    "globalIdLocal": 1050200,
    "local": "Castelo Branco",
    " "latitude": "39.8217",
    "longitude": -7.4957




  }]
}
```

→

DTO com `kotlinx.serialization`

```kotlin
@Serializable
data
    class RespostaLocais(
    val data: List<Local>,

)

@Serializable
data class Local(
    val glocalIdLocal:Int,
    val local: String,          // coordenadas como texto!
    val latitude: String,
    val longitude: String,

)

val json = Json { ignoreUnknownKeys = true }
```

Campos a mais (`owner` , `idRegiao` ...) → `ignoreUnknownKeys` , senão falha.

Coordenadas em texto (`"40.6413"`) → ficam String

Bibliotecas: kotlinx.serialization (sem reflexão) · Moshi · Gson (reflexão, Java, em manutenção).

GraphQL: o cliente escolhe os campos

`POST /graphql` — um único endpoint

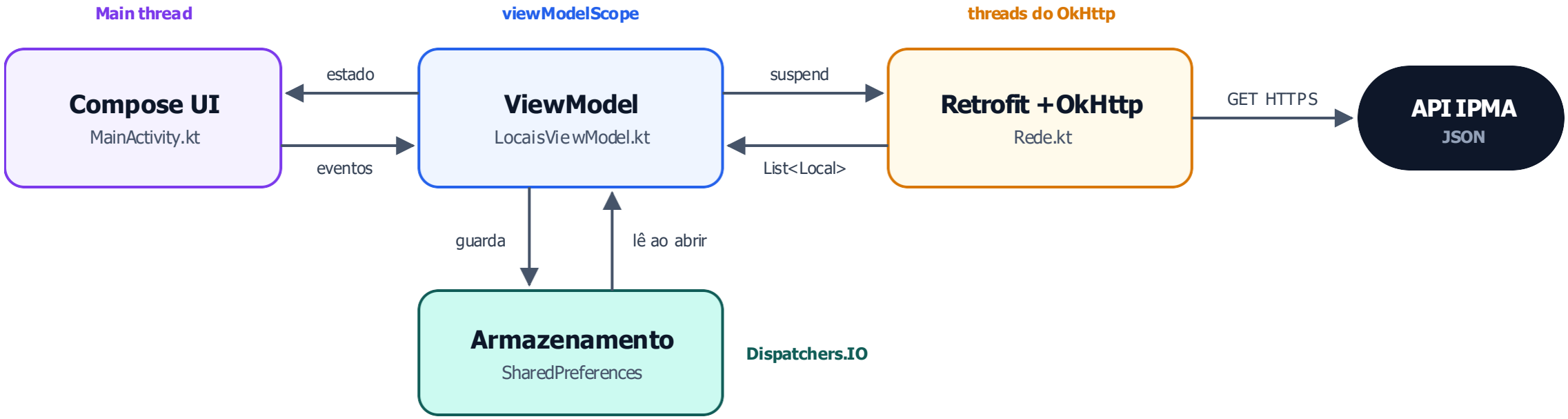
```
query PaisesEuropa {
  continent(code: "EU") {
    countries {
      code
      name
      capital
    }
  }
}
```

```
{ "data": { "continent": { "countries": [
    { "code": "PT", "name": "Portugal",
      "capital": "Lisbon" }, ...
] } } }
```

	REST	GRAPHQL
Endpoints	um por recurso	um só (`/graphql`)
Forma da resposta	definida pelo servidor	definida pela query
Dados a mais / a menos	frequente (vários pedidos)	pede-se só o necessário
Cache HTTP	simples (GET + cabeçalhos)	POST não usa cache HTTP padrão → cache no cliente (Apollo)
Erros	código HTTP	campo `errors` , muitas vezes com 200
Tipos	OpenAPI (opcional)	schema obrigatório

> Em Android: **Apollo Kotlin** lê o schema, gera classes tipadas (`PaisesEuropaQuery`) e expõe funções `suspend` e `Flow` — o resto da arquitetura não muda.

Arquitetura: cada camada com uma responsabilidade



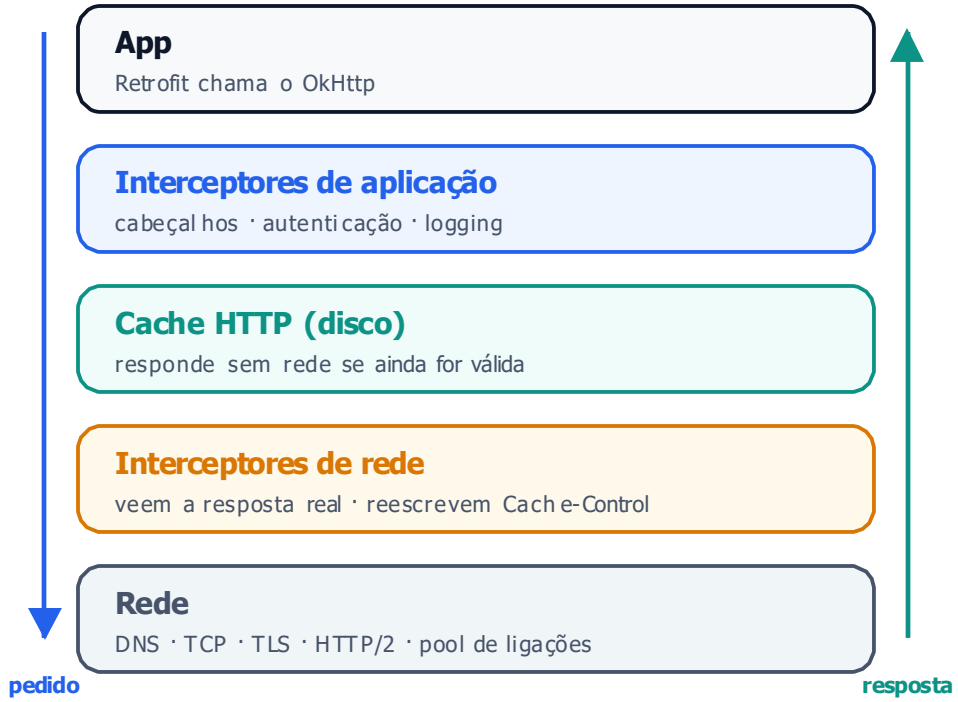
UI só desenha o estado e envia eventos. Não sabe o que é HTTP.

ViewModel sobrevive à rotação do ecrã e cancela o trabalho em `onCleared()`.

Armazenamento guarda a última resposta → a app abre **sem rede**. Em apps maiores: repositório + Room.

OkHttp: o cliente HTTP por baixo de tudo

```
val cliente = OkHttpClient.Builder()
    .connectTimeout(10, TimeUnit.SECONDS)
    .readTimeout(15, TimeUnit.SECONDS)
    .callTimeout(20, TimeUnit.SECONDS)
    .cache(Cache(File(cacheDir, "http"), 10L * 1024 * 1024))
    .addInterceptor { chain ->                    // aplicação
        chain.proceed(chain.request().newBuilder()
            .header("Accept", "application/json")
            .build())
    }
    .addNetworkInterceptor(cacheDezMinutos)       // rede
    .addInterceptor(HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BASIC
    })
    .build()    // criar UMA vez e reutilizar
```



Por omissão: **connect, read e write = 10 s**; **call = sem limite**. A demo configura só os timeouts.

Um cliente por app — cada instância tem o seu pool de ligações, threads e cache.

Interceptor de aplicação corre 1× por chamada; o **de rede** corre por cada ida real à rede.

Logging só em debug: `BODY` expõe tokens e dados pessoais nos logs.

Retrofit: a API descrita como uma interface

```
interface IpmaApi {
    @GET("open-data/distrits-islands.json")
    suspend fun obterLocais(): RespostaLocais      // usado na demo

    @GET("open-data/forecast/meteorology/cities/daily/{id}.json")
    suspend fun previsao(@Path("id") id: Int): Previsao    // exemplo de @Path
}

val conversor = json.asConverterFactory("application/json".toMediaType())

val api: IpmaApi = Retrofit.Builder()
    .baseUrl("https://api.ipma.pt/")          // termina em "/"
    .client(cliente)                          // o OkHttp do slide anterior
    .addConverterFactory(conversor)           // JSON ⇄ objetos
    .build()
    .create(IpmaApi::class.java)              // implementação gerada
```

ANOTAÇÃO	EFEITO
`@GET @POST…`	método + caminho
`@Path`	segmento do URL
`@Query`	`?chave=valor`
`@Body`	objeto → JSON
`@Header`	cabeçalho

suspend → o Retrofit faz o pedido fora da main thread: é **main-safe**.

Devolve `T` (lança `HttpException` em 4xx/5xx) ou `Response<T>` para ver código e cabeçalhos.

Onde corre cada coisa

Três "pistas" de execução

`Main`
desenhar · toques · atualizar estado da UI

`Dispatchers.IO`
rede · ficheiros · base de dados (bloqueante)

`Dispatchers.Default`
CPU: ordenar, parsing pesado, imagens

A main thread é **uma só**. Bloqueá-la = frames perdidos;

Ferramentas, da mais antiga à atual

- ~~Thread + Handler~~ — manual, fácil de criar fugas
- ~~AsyncTask~~ — obsoleto desde a API 30
- **ExecutorService** — pools de threads em Java
- **RxJava** — streams reativos, curva íngreme
- **Coroutines + Flow** — **recomendado em Kotlin**

```
// CPU → Default
suspend fun ordenar(lista: List<Local>) =
    withContext(Dispatchers.Default) { lista.sortedWith(ordem) }

// disco / rede bloqueante → IO
suspend fun lerCache(f: File) =
    withContext(Dispatchers.IO) { f.readText() }
```

Com coroutines, mudar de "pista" é uma linha: `withContext(…)`. Quem chama não precisa de saber — a função é **main-safe**.

Coroutines: código sequencial, execução assíncrona

```kotlin
// LocaisViewModel.kt (versão resumida)
fun carregar() {
    viewModelScope.launch {          // Main; cancelada em onCleared()
        _estado.update { it.copy(aCarregar = true, erro = null) }
        try {
            val resposta = Rede.api.obterLocais()    // suspende; UI fluida
            armazenamento.guardarLocais(resposta.data)
            _estado.update { it.copy(locais = resposta.data) }
        } catch (e: CancellationException) {
            throw e                              // cancelar não é erro
        } catch (e: Exception) {
            _estado.update { it.copy(erro = mensagemDeErro(e)) }
        } finally {
            _estado.update { it.copy(aCarregar = false) }
        }
    }
}
```

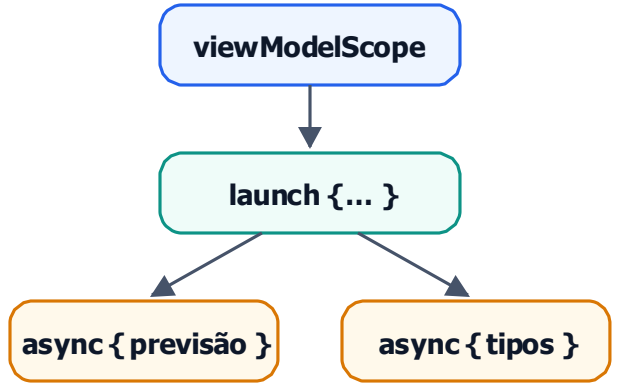
`suspend`
função que pode pausar e retomar sem prender a thread

`CoroutineScope`
define o tempo de vida: `viewModelScope` , `lifecycleScope`

`Dispatcher`
onde corre: Main · IO · Default

`Job`
referência à coroutine — pode ser cancelada

Concorrência estruturada



Pedidos em paralelo, limites de tempo e cancelamento

```
suspend fun atualizarPrevisao(id: Int): Unit = coroutineScope {
    val previsao = async { api.previsao(id) }
    val tipos    = async { api.tiposTempo() }    // ao mesmo tempo
    guardar(previsao.await(), tipos.await())
}   // se um falhar, o outro é cancelado e o erro propaga-se

// Tempo máximo do lado da coroutine
val r = withTimeoutOrNull(5_000) { api.obterLocais() } // null após 5 s
withTimeout(5_000) { … }    // lança TimeoutCancellationException
```

	DEVOLVE	USO
`launch`	`Job`	"faz isto" — sem resultado
`async`	`Deferred<T>`	resultado com `await()`



valores ilustrativos

O cancelamento é **cooperativo**: sai-se no próximo ponto de suspensão. Nunca apanhar `CancellationException` sem a relançar.

`viewModelScope` — cancelado quando o ViewModel é destruído (`onCleared`).

`lifecycleScope` — ligado à Activity/Fragment; recolher com `repeatOnLifecycle`.

`GlobalScope` — **evitar**: nunca é cancelado, provoca fugas de trabalho e memória.

Em Java: ExecutorService + LiveData

```java
ExecutorService executor = Executors.newFixedThreadPool(2);
MutableLiveData<List<Local>> locais = new MutableLiveData<>();

void carregar() {
    aCarregar.setValue(true);                    // main thread → setValue
    executor.execute(() -> {                     // thread do pool
        try {
            Response<RespostaLocais> r = api.obterLocais().execute();
            if (r.isSuccessful()) locais.postValue(r.body().data);
            else erro.postValue("Erro HTTP " + r.code());
        } catch (InterruptedIOException e) {
           erro.postValue("O servidor demorou demasiado")
       } catch(IOException e){
           Erro.postValue("Sem ligação á internet") ;
        } finally {
    });          aCarregar.postValue(false);          // outra thread → postValue
}       }

@Override protected void onCleared() { executor.shutdownNow(); }
```

Pool de threads

`newFixedThreadPool(n)` reutiliza threads em vez de criar uma por pedido.

Voltar à main thread

`postValue()` do LiveData ou `Handler(Looper.getMainLooper()).post(…)` .

Alternativa curta

`call.enqueue(callback)` : o Retrofit já executa em fundo e devolve o callback na main thread.

Ciclo de vida

`shutdownNow()` em `onCleared()` — senão as threads sobrevivem ao ecrã.

LiveData ou Flow

	LIVEDATA	STATEFLOW / FLOW
Linguagem	Java e Kotlin	Kotlin (coroutines)
Ciclo de vida	consciente por natureza	`collectAsStateWithLifecycle`
Valor inicial	opcional	obrigatório (StateFlow)
Operadores	`map`, `switchMap`	`map`, `combine`, `debounce`, `flatMapLatest` …
Threads	`setValue` / `postValue`	`flowOn`, dispatchers
Fora do Android	não	sim (dados, testes JVM)

```kotlin
// ViewModel: só o ViewModel altera o estado
private val _estado = MutableStateFlow(EstadoUi())
val estado: StateFlow<EstadoUi> = _estado.asStateFlow()

// alterar = criar uma cópia com os campos novos
_estado.update { it.copy(aCarregar = true) }

// Compose: recolhe só com o ecrã visível
// e redesenha sempre que o estado muda
val estado by vm.estado.collectAsStateWithLifecycle()
```

Projeto novo em Kotlin → **Flow**. LiveData continua válido em Java e em código existente. Na demo: **StateFlow**.

Estado da UI: o ecrã é uma função do estado

```
data class EstadoUi(
    val locais: List<Local> = emptyList(),
    val favoritos: Set<Int> = emptySet(),
    val aCarregar: Boolean = false,
    val erro: String? = null,
    val atualizadoEm: Long = 0L,
)
```

Com cache, dados, carregamento e erro **coexistem** — por isso uma `data class` e não só `Loading | Success | Error`. O `sealed interface` serve bem ecrãs sem cache, onde os estados se excluem.



1.º carregamento — vazio + a carregar

Sucesso — lista

A atualizar — lista + a carregar

Erro sem dados — ecrã de erro

Erro com cache — lista + aviso

Tratamento de erros: cada falha tem a sua resposta

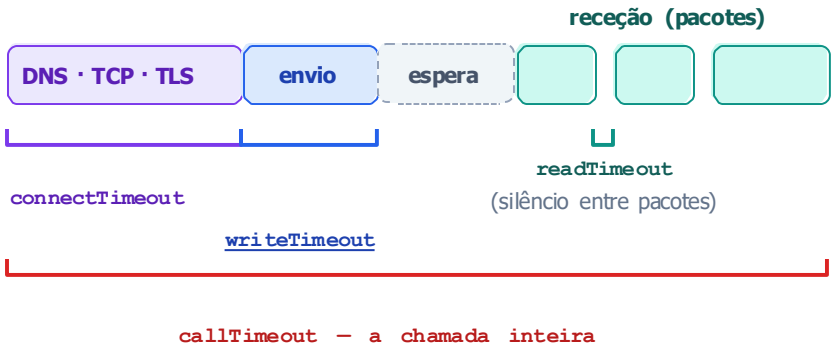
EXCEÇÃO	CAUSA TÍPICA	O QUE A UI MOSTRA	REPETIR?
`UnknownHostException` · `ConnectException`	sem rede, DNS, servidor em baixo	"Sem ligação" + dados em cache	**sim**
`SocketTimeoutException` · InterruptedException	rede lenta, servidor sobrecarregado	"Demorou demasiado"	**sim, com backoff**
`HttpException` 4xx	pedido inválido, 401/403, 404	mensagem específica	**não (429: esperar)**
`HttpException` 5xx	erro do servidor	"Problema no servidor"	**sim, com limite**
`SerializationException`	JSON mudou / campo em falta	mensagem genérica + log	**não**
`CancellationException`	o utilizador saiu do ecrã	nada — **relançar**	**não**

```
fun mensagemDeErro(e: Exception): String = when (e) {
    is InterruptedIOException -> "O servidor demorou demasiado."
    is IOException            -> "Sem ligação à internet."
    is HttpException          -> "Erro do servidor (HTTP ${e.code()})."
    is SerializationException -> "Formato inesperado."
    else                      -> "Erro inesperado."
}   // a ordem conta: o timeout é um tipo de IOException
```

Traduzir as exceções técnicas num **único sítio**. O ecrã mostra a mensagem por cima da lista guardada, com o botão "Tentar de novo".

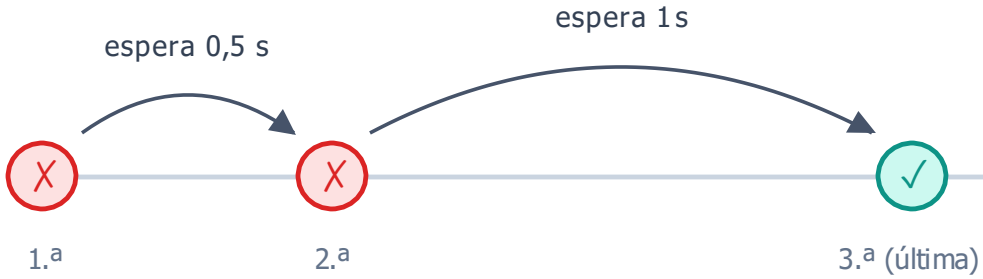
Tempos de espera e repetição com backoff

Os quatro timeouts do OkHttp



	CONNECT	READ	WRITE	CALL
omissão	10 s	10 s	10 s	sem limite
demo	10 s	15 s	10 s	20 s

Repetição com backoff exponencial



Só pedidos **idempotentes** (GET, PUT, DELETE)

Número de tentativas **limitado**; esperas a duplicar (+ aleatoriedade em produção)

Em **429/503**, respeitar o cabeçalho `Retry-After`

Do lado das coroutines: `withTimeout` / `withTimeoutOrNull`

Cache: mostrar já, atualizar depois

Memória — **StateFlow no ViewModel**
sobrevive à rotação; perde-se ao fechar a app

↓ falha / expirou

Disco — **SharedPreferences (demo) · Room**
sobrevive ao fecho da app e à falta de rede

↓ falha / expirou

Cache HTTP — **OkHttp Cache (opcional)**
Cache-Control: max-age · ETag → 304 Not Modified

↓ falha / expirou

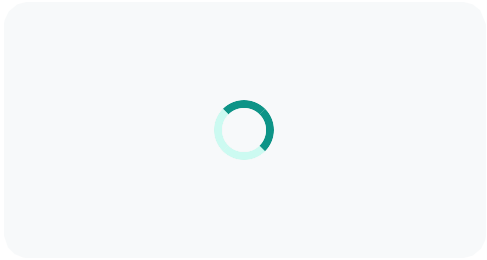
Rede — **API do IPMA**
lenta, pode falhar, pode custar dados móveis

O que faz a demo: mostrar já, atualizar depois

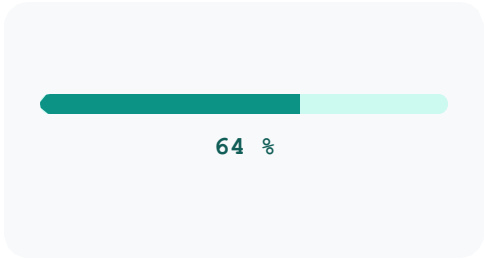
1. Ao abrir, **lê o que está guardado** e mostra logo a lista
2. Pede a lista nova à **API**
3. Se chegar: **guarda** no telemóvel (JSON + hora) e atualiza o ecrã
4. Se falhar: a lista antiga **fica** + "Sem ligação" + "Atualizado em…"
5. Os **favoritos ★** também ficam guardados

Em apps maiores: **Room** como fonte única de verdade e **cache HTTP** do OkHttp para não repetir pedidos em poucos minutos.

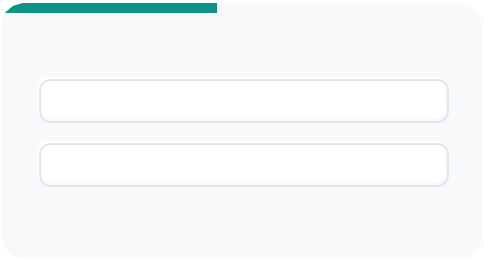
Indicadores de progresso: dizer ao utilizador o que se passa



Indeterminado
duração desconhecida — 1.º carregamento



Determinado
progresso mensurável — downloads, uploads



Pull-to-refresh / barra linear
atualizar com dados já visíveis

Skeleton
esboço do layout enquanto carrega

Nunca bloquear o ecrã inteiro para atualizar dados que já estão visíveis.

Frescura visível: "Atualizado há 5 min" quando se mostra cache.

Evitar duplicados: ignorar "atualizar" enquanto um pedido está em curso.

Material 3 / Compose: `CircularProgressIndicator`, `LinearProgressIndicator`, `PullToRefreshBox`, `Snackbar`.

SÍNTESE

Vantagens e desvantagens

VANTAGENS

✓ **Coroutines**
Concorrência estruturada: menos fugas e integração com o Jetpack

✓ **Funções main-safe**
Chamadas a partir da UI sem a bloquear com a rede

✓ **ViewModel**
Guarda o estado e mantém-no nas rotações do ecrã

✓ **OkHttp**
Reutiliza a ligação entre pedidos ao mesmo servidor

DESVANTAGENS

✗ **SharedPreferences**
Não sinaliza erros de escrita nem tem API transacional

✗ **ViewModel**
Perde o estado se o sistema matar o processo →SavedStateHandle

✗ **OkHttp**
Não permite GET com corpo; a cache HTTP só cobre GET

✗ **Retrofit com R8**
O R8 pode remover tipos genéricos dos métodos →exige regras extra

EM RESUMO

1

HTTP +REST +JSON

o contrato com o servidor. Retrofit e OkHttp tratam da canalização.

2

Coroutines +Flow

a UI nunca espera pela rede; o estado chega quando chega.

3

Erros, timeouts, cache

a rede vai falhar — a app tem de continuar útil.

PERGUNTAS

P1 **Se o pedido à API for feito na main thread, o que acontece? Como se resolve em Kotlin?**

P2 **A app abre em modo de avião. O que deve aparecer no ecrã — e o que é preciso para isso funcionar?**